

第 10 讲实验设计

从手算 MLP 到 NumPy / PyTorch 一致性验证

- 核心原则：先预测，再运行，再解释差异。
- 本实验不是追求准确率，而是验证计算机制。

学习方法：先看懂每一步数字和 Shape，再让 NumPy / PyTorch 帮我们自动计算。

1. 实验问题与假设

研究问题	可检验假设
手算的 3->4->2 MLP 能否被 NumPy 精确复现？	相同参数下，输出应为 [2.7,0.2]
每个隐藏神经元是否等价于一次 dot product + bias？	逐神经元结果应与矩阵乘法的对应位置一致
参数量公式是否正确？	手算 26，应与 PyTorch parameter numel 总和相同
Batch/3D Tensor 是否只是把相同 MLP 应用到更多样本 /Token？	最后一维按 3->4->2 变化，B/T 保留
NumPy 与 PyTorch 是否实现同一数学？	复制相同参数后，输出差异只应来自浮点误差

2. 实验变量与控制

本实验不是训练实验，因此没有“准确率”这个指标。我们关心的是机制一致性。

项目	设计
固定输入	$x=[1,2,-1]$
固定参数	$W1,b1,W2,b2$ 在 NumPy 与 PyTorch 中完全相同
激活函数	ReLU
核心指标	输出数值、最大绝对误差、Shape、参数量
控制变量	不训练、不随机初始化比较；PyTorch 权重直接复制

为什么这样设计？

如果 NumPy 和 PyTorch 使用不同随机权重，就无法判断差异来自“框架”还是“参数”。因此这里把参数固定，形成最小可解释实验。

3. 实验步骤 A：单样本 Forward

```
z1 = x @ W1 + b1
a1 = relu(z1)
y = a1 @ W2 + b2
```

运行前先预测：z1、a1、y 的 Shape 和数值。然后再运行脚本，核对是否得到：

```
z1 = [1, 3, -1, -3]
a1 = [1, 3, 0, 0]
y = [2.7, 0.2]
```

如果结果不同，不要先怀疑 NumPy；先逐项检查矩阵方向、Bias 和 ReLU。

4. 实验步骤 B：隐藏神经元拆解 + 参数量

脚本会逐个取 W1 的列向量，计算 $x \cdot w$ ，再求和并加 Bias。目的不是提高效率，而是把一次矩阵乘法还原成四个神经元。

```
for j in range(W1.shape[1]):  
    contribution = x * W1[:, j]  
    neuron_z = contribution.sum() + b1[j]
```

随后手算参数量并与代码统计对照：

```
■■■: 3*4 + 4 = 16  
■■■: 4*2 + 2 = 10  
■■■: 26
```

5. 实验步骤 C：Batch 与 LLM 三维 Tensor

先输入 $X.shape=(4,3)$ ，再输入 $X3.shape=(2,3,3)$ 。观察最后一维如何按 $3 \rightarrow 4 \rightarrow 2$ 变化。

```
X.shape = (4, 3)
Z1.shape = (4, 4)
Y.shape = (4, 2)
```

```
X3.shape = (2, 3, 3)
Z13.shape = (2, 3, 4)
Y3.shape = (2, 3, 2)
```

特别检查 $X3[0,0]$ 与实验 1 的 x 相同，因此 $Y3[0,0]$ 应与实验 1 的 y 相同。这个检查能验证三维批量处理没有改变单 Token 的 MLP 规则。

6. 实验步骤 D：PyTorch 对照

用 `nn.Sequential` 构造同一结构，并把 NumPy 权重复制进去。注意 PyTorch Linear 的 `weight` 存储顺序是 `(out_features, in_features)`，所以复制时需要转置。

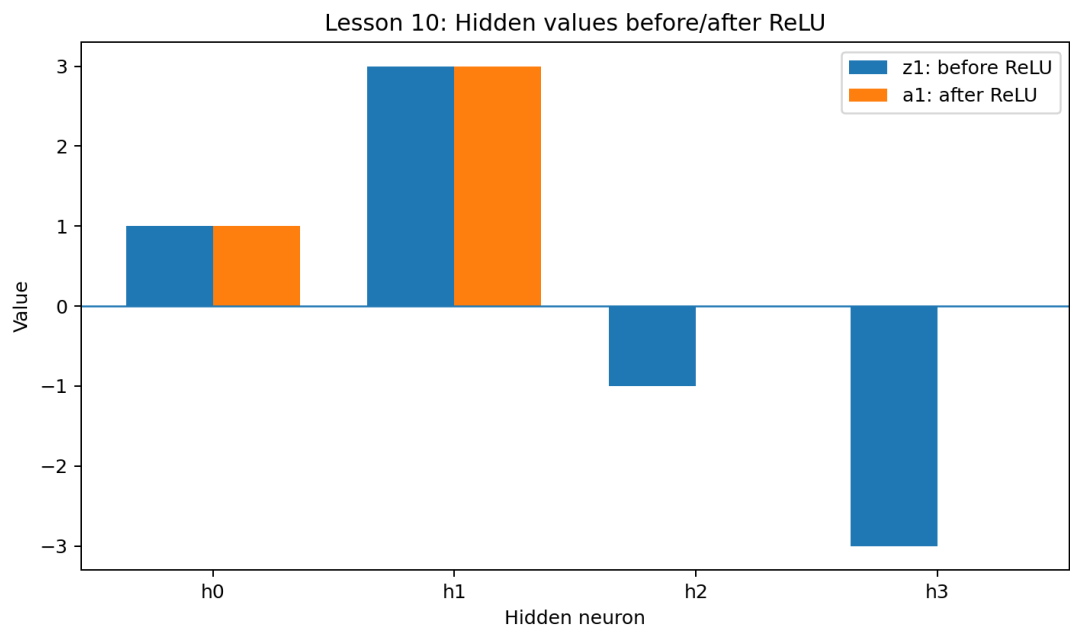
```
model = nn.Sequential(  
    nn.Linear(3,4),  
    nn.ReLU(),  
    nn.Linear(4,2),  
)  
  
model[0].weight.copy_(torch.tensor(W1.T))  
model[2].weight.copy_(torch.tensor(W2.T))
```

验收：NumPy 与 PyTorch 最大绝对误差应接近浮点精度；参数量应为 26。

7. 结果记录与解释模板

项目	实际结果	解释
单样本输出	[2.7,0.2]	手算与 NumPy 一致
参数量	26	与公式一致
3D Shape	(2,3,3)->(2,3,4)->(2,3,2)	B/T 不变，最后一维变化
PyTorch 最大误差	约 4.8e-08	浮点精度差异

实验图 lesson10_hidden_activation.png 会展示 z1 与 ReLU(z1) 的差异：负的隐藏值被截为 0，正值保持。



8. 实验后的科研思维

本实验最重要的不是“会写 `nn.Sequential`”，而是形成一个习惯：任何复杂网络结构，都优先拆成可验证的中间变量和 Shape。

以后做 Agent Systems 研究，即使你不训练基础模型，也经常需要判断某个 embedding、projection、adapter、router、scorer 或 evaluation head 的实现是否合理。这个“拆计算图 + 查 Shape + 做最小对照”的能力会反复使用。

实验完成标准

你可以不看答案重新写出 NumPy Forward；能够说明为什么 PyTorch weight 要转置复制；能够解释为什么 3D Tensor 中 B 和 T 不因普通 MLP 而改变。